Typst のすゝめ

実験レポートのための入門書

電子工学科 4 年

1. 目的

実験レポートは数式・図表・参考文献が多く，番号の付け方や配置にも決まりがある。本クラスでは主に Word が使われており，LaTeXを使う人も増えつつある。そこで本書では，実験レポートの作成ツールとして，Word やLaTeXに加えて Typst を提案する。そのために，コンパイル時間，記法，機能を Word とLaTeXの両方と比較して示し，導入の方法も示す。

本書自体も, 学科の「実験報告書 (レポート) の書き方」の様式 (A4 縦, 本文は明朝体, 見出しはゴシック体，図の見出しは図の下，表の見出しは表の上など) に合わせて Typst で組んでいる。本書のソース一式は GitHub(`https://github.com/i0roche/typst-no-susume`) で公開しており，書式をまとめた `common.typ` は，そのまま実験レポートに流用できる。

2. Typst とは

Typst は Rust で実装された新しい組版システムである[1]。文章をテキストファイルに書き, コンパイルすると PDF が得られる。この流れはLaTeXと同じだが, LaTeXと同程度の表現力を，より簡単な使い方で実現することを目指している[1]。無料のオープンソースソフトウェア[1]であり, ブラウザ上の Web アプリ (typst.app) でも，パソコンに入れるコマンドラインツールとしても使える。

書き方には 3 つの場面がある。文章は Markdown に近い記法で書き (`=` で見出し，`-` で箇条書き), 数式は `$` で囲んで書き, プログラムは `#` から始めて書く。この 3 つは 1 つのファイルの中で切り替えながら使える。文章を書く感覚のまま，必要なところだけ数式やプログラムを足していける。

3. 特長

(1) **コンパイルが軽い** Thoughtworks の技術レーダーでは，LaTeXより高速なコンパイルが評価され, 採用を試す価値がある「Trial」に位置づけられている[2]。手元の測定では, 同じ文書の全体のコンパイルが, 学校のテンプレートと同じpLaTeXより約 1.6〜1.8 倍, LuaLaTeXより約 11〜13 倍速かった。1 か所を直したあとの再コンパイルは約 0.012 秒で終わった (後述)。入力のたびに結果を確かめる書き方をしても，待たされない。

(2) **VS Code で即プレビュー** 拡張機能 Tinymist は言語サーバとプレビューを提供する[3]。ソースの横にプレビューを表示でき，入力に合わせて更新される。実験レポートでの使用例でも，ほぼ遅延なく更新されたと報告されており，LaTeXでは保存してから 3〜10 秒かけてコンパイルしていたという[4]。

(3) **エラーが読みやすい** LaTeXより明確なエラーメッセージを出すと評価されている[2]。図 1 のように，ファイル名・行・列と原因を示し，ソースの該当箇所を指してくれる。メッセージは英語だが, どこを直せばよいかはすぐ分かる。同じ間違い (存在しない参照) をLuaLaTeXでコンパイルすると，図 2 のように，行番号つきの警告が出るだけで，エラーにならず PDF が作られる。本文中には「??」が出力され，ログの中から警告を探す必要がある。

[1] ライセンスは Apache-2.0 である。

(4) **数式を短く書ける** 分数は `a/b`，平方根は `sqrt(x)`，ギリシャ文字は `pi` のように，名前をそのまま書く。TEXの `\frac{a}{b} \sqrt{x} \pi` と比べて短い[2]。

(5) **中間ファイルが出ない** LATEXでは `.aux .log .toc` などの補助ファイルができる[3]。Typst が出力するのは PDF だけで，フォルダにはソースと結果だけが残る。

(6) **関数で自動化できる** 変数，条件分岐，繰り返し，関数を使える。有効数字の整形や表の生成のような，実験レポートの面倒な処理をプログラムとして書ける。有効数字を揃える関数を自作した例がある[4]。

(7) **日本語を書きやすい** 日本語のフォントと `lang: "ja"` を指定すれば，日本語を書ける。LATEXのように，日本語用のクラスやエンジンを選ぶ必要がない。`lang` を指定しないと，図表の見出しが「Table 1」のように英語になる。また，ソースで和文の途中を改行すると，出力に半角スペースが入る。段落の中では改行しないか，パッケージ cjk-unbreak で取り除く[5]。

4. コンパイル時間の測定

同じ内容の文書を Typst，pLATEX，LuaLATEXで書き，コンパイル時間を比べた。pLATEXは学校のレポートのテンプレート (jarticle) が前提とするエンジンで，LuaLATEXは現在よく勧められるエンジンである。文書は日本語の本文，数式，表，図を含む A4 の文書で，章の数を 2・12・36 に変えて 3 種類作った[4]。結果を表 1 に示す。

全体のコンパイルでは，Typst はpLATEXより約 1.6〜1.8 倍，LuaLATEXより約 11〜13 倍速かった。pLATEXも 36 章で約 0.38 秒であり，差は小さい。LATEXの速さはエンジンによって大きく変わり，遅いのは主にLuaLATEXである。

再コンパイルは 1 か所を直したあとにもう一度コンパイルする場面であり，プレビューを見ながら書くときの待ち時間に当たる。LATEXは，参照が変わらなければ 1 回の処理で済み，pLATEX[5]は約 0.12 秒，LuaLATEXは約 0.91 秒かかった。Typst は前回の結果を再利用するので，`typst watch` でのコンパイル自体は約 0.012 秒だった。ただし保存から PDF の更新までは約 0.1 秒で，体感の差は小さい。強みは，入力のたびにプレビューが更新される点にある。

LATEXの全体のコンパイルは，参照を解決するために，latexmk が 2 回処理した時間である。Typst は 1 回の実行で参照を解決する。Typst 0.15.1，pLATEX(jarticle) と dvipdfmx，LuaHBTeX 1.24.0(ltjsarticle)，いずれも TEX Live 2026 で，筆者の PC(Linux) で測った。Typst は 10 回 (再コンパイルは 8 回)，LATEXは各 5 回の中央値を示した。厳密なベンチマークではない。結果は文書の内容や環境によって変わる。

表 1　同じ文書のコンパイル時間

場面	Typst	pLATEX	LuaLATEX
全体 (2 章)	約 0.14 秒	約 0.25 秒	約 1.8 秒
全体 (12 章)	約 0.17 秒	約 0.29 秒	約 2.0 秒
全体 (36 章)	約 0.23 秒	約 0.38 秒	約 2.6 秒
再コンパイル (12 章)	約 0.012 秒	約 0.12 秒	約 0.91 秒

[2]筆者の所感であり，TEXに慣れていればTEXの方が書きやすいという意見もある。

[3]手元の測定では，LuaLATEXは `.aux .log .fls .fdb_latexmk` を生成した。

[4]ページ数は Typst で 3・13・37，pLATEXで 3・16・46，LuaLATEXで 3・14・41 である。12 章の文書は数式 12 個，表 12 個，図 4 枚を含む。

[5]dvipdfmx での PDF 化の時間を含む。

```
error: label `<eq0で表される>` does not exist in the document
   ┌─ bench.typ:10:7
   │
10 │ 共振周波数は式@eq0で表される。
   │                ^^^^^^^^^^^^^^
```

図 1　Typst のエラーメッセージの例 (存在しないラベルの参照)

```
LaTeX Warning: Reference `eq0' on page 1 undefined on input line 3.
LaTeX Warning: There were undefined references.
```

図 2　LuaLaTeXの出力の例 (存在しない参照，エラーにならない)

5. Word・LaTeX・Typst の比較

三者の特徴を表 2 にまとめる。評価は筆者の所感である。

Word は見た目を確かめながら直接編集でき，導入の障壁が最も低い。一方，数式の入力や図表番号・相互参照の管理にマウス操作が多く，書式をそろえる作業が手作業になりやすい。LaTeXは組版の品質，数式，文献管理の点で定番であり，学会のテンプレートなどの資産も多い。一方，環境の構築が重く，エンジンによってはコンパイルに数秒かかり (表 1)，エラーメッセージが読みにくい。Typst はLaTeXと同じく文章をテキストで書く方式を，軽さと書きやすさで改良したものと位置づけられる。一方，資産はまだ少なく，縦書きやルビには外部パッケージが必要である。

6. 記法の比較 (LaTeX と Typst)

LaTeXは `\` から始まる命令と，`{ }` で囲む引数，`\begin` と `\end` で囲む環境で書く。Typst は見出しや箇条書きを記号で書き，命令は `#` から始める。同じことを書いたときの記法を表 3 に示す。

実際に同じ内容を書くと，図 3 と図 4 のようになる。Typst の方が行数も記号も少なく，ソースがそのまま読みやすい。

表 2　三者の比較

	Word	LaTeX	Typst
導入の手軽さ	◎	△ 環境構築	○ Web 版あり
待ち時間	◎	○〜△ エンジンによる	◎ 速い
エラーの分かりやすさ	—	△	○
数式	△ GUI 操作	◎ 定番	◎ 短い
図表・文献の番号	△ 手作業	◎	◎
Git での差分管理	△	◎	◎
学会テンプレート	○	◎	△ 少ない
縦書き・ルビ	◎	◎	△ パッケージ

表 3　同じことを書くときの記法

	LaTeX	Typst
見出し	`\section{目的}`	`= 目的`
強調	`\textbf{重要}`	`*重要*`
箇条書き	`\begin{itemize}\item …`	`- …`
分数	`$\frac{a}{b}$`	`$a/b$`
図の参照	`\ref{fig:a}`	`@fig-a`
コンパイル	`latexmk`	`typst compile`

(同じ内容の比較)

```latex
\section{目的}
ラインセンサの出力を\textbf{調べる}。
\begin{itemize}
  \item 白: 0.8 V 以上
  \item 黒: 0.2 V 以下
\end{itemize}
```

図 3　LaTeXで書いた例

```
= 目的
ラインセンサの出力を*調べる*。
- 白: 0.8 V 以上
- 黒: 0.2 V 以下
```

図 4　Typst で書いた例

7. 数式

数式は `$` で囲んで書く。前後に空白を入れると独立行になり，中央に置かれて，右端に式番号が付く。空白を入れないと，文中の数式になる。分数は `/`，上付きは `^`，下付きは `_`，平方根は `sqrt()`，総和は `sum` のように，TEXより短い名前で書く。ギリシャ文字は `pi` や `alpha` と書くだけでよい。単位などの文字は引用符で囲むと立体になり，物理量を斜体，単位を立体とする規則に合わせられる。次の例では共振周波数を式(1) として書いている。

(数式の例)

```
共振周波数 $f_0$ は，次式で表される。
$ f_0 = 1/(2 pi sqrt(L C)) quad ["Hz"] $ <eq-res>
ここで，$L$ はインダクタンス，$C$ は静電容量である。式@eq-res より，$f_0$ は $L C$ の平方根に反比例することがわかる。
```

共振周波数 f_0 は，次式で表される。

$$f_0 = \frac{1}{2\pi\sqrt{LC}} \quad [\mathrm{Hz}] \tag{1}$$

ここで，L はインダクタンス，C は静電容量である。式(1) より，f_0 は LC の平方根に反比例することがわかる。

式番号は自動で振られ，`@eq-res` のように書けば本文から引用できる。式を追加して番号がずれても，引用の番号も一緒に更新される。注意点として，複数の文字を続けて書くと 1 つの名前として扱われる。そのため，L と C の積は `LC` ではなく `L C` と，空白で区切って書く。

行列は `mat()`，総和は `sum` で書ける。式(2) はその例である。

(行列と総和の例)

```
$ A = mat(1, 2; 3, 4), quad
  sum_(k=1)^n k = (n(n+1))/2 $ <eq-mat>
```

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \sum_{k=1}^{n} k = \frac{n(n+1)}{2} \tag{2}$$

8. 表と図表番号

表の見出しは表の上に付き，番号は自動で振られる。本文からは 表 4 のように引用でき，表の順序を入れ替えても番号を手で直す必要がない。罫線は `table.hline()` で引き，三線表にしている。

(表の例)

```
- 白: 0.8 V 以上
- 黒: 0.2 V 以下

#figure(
  table(columns: 2,
    rows: (2pt, auto),
    table.hline(),
    table.cell(colspan: 2)[],
    table.hline(),
    [状態], [出力 [V]],
    table.hline(),
    [白], [0.85],
    [黒], [0.12],
    table.hline()),
  caption: [センサ出力],
) <tab-sensor>

@tab-sensor を参照。
```

- 白: 0.8 V 以上
- 黒: 0.2 V 以下

表 4　センサ出力

状態	出力 [V]
白	0.85
黒	0.12

表 4 を参照。

図も同じ仕組みで，画像ファイルを `image()` で読み込み，`figure()` で囲んで見出しとラベルを付ける。見出しは図の下に付き，番号は表とは別に数えられる。図 5 に例を示す。

```
#figure(
  image("circuit.png", width: 60%),
  caption: [回路図],
) <fig-circuit>

回路を@fig-circuit に示す。
```

図 5　図の読み込みと引用

9. 書式の設定

Typst では文書全体の書式を `#set` と `#show` で宣言する。`#set` はページ，文字，段落，見出しなどの既定値を決める。`#show` は特定の要素の見た目を決める。図 6 は見出しの番号，式番号，段落の 1 字下げ，表の見出しを表の上に置く設定である。このような決まりを数行で書ける。

```
#set page(paper: "a4", numbering: "1")
#set text(font: "Harano Aji Mincho", size: 10.5pt, lang: "ja")
#set par(first-line-indent: (amount: 1em, all: true))
#set heading(numbering: "1.")
#set math.equation(numbering: "(1)")
#show figure.where(kind: table): set figure.caption(position: top)
```

図 6　書式の設定の例

書式の決まりをこのように 1 か所にまとめておけば，レポートごとに設定し直す必要がなく，書式のばらつきも防げる。本書もこの方法で書式を `common.typ` にまとめている。

10. 関数による自動化

Typst には変数，条件分岐，繰り返し，関数がある。データの丸めや表の生成のような繰り返し作業を，関数にして使い回せる。有効数字を揃える関数を自作して，表の全数値に適用した例が報告されている[4]。

(関数の例)

```
#let r3(x) = str(calc.round(x, digits:
3))
#let data = (1.23456, 2.5, 3.14159)
#for x in data [
  - #x → #r3(x)
]
```

- 1.23456 → 1.235
- 2.5 → 2.5
- 3.14159 → 3.142

測定データは CSV ファイルから読み込める。図 7 のように `csv()` で読み込めば，データを書き写す手間と，転記ミスを減らせる。作図や SI 単位の表記には，パッケージ cetz や unify がある[6]。パッケージは 1 行で読み込め，インストールの作業は要らない。

```
#let rows = csv("data.csv")
#table(columns: 2, ..rows.flatten())
```

図 7　CSV ファイルの読み込み

表の罫線のように，毎回同じ形になる書き方も関数にまとめられる。図 8 は筆者が自作した三線表の関数 `ftab` である。表題，見出し行，データを渡すだけで，罫線と番号の付いた表ができる。表を組むたびに `table.hline()` を並べる手間がなくなり，罫線の太さを変えたいときも 1 か所の修正で済む。

```
#let ftab(capt, header, cells, sup: "表") = {
  [#figure(
    caption: capt,
    supplement: sup,
    table(
      columns: header.len(),
      rows: (2pt,) + (auto,),
      table.hline(),
      table.cell(colspan: header.len())[],
      table.hline(),
      ..header.flatten(),
      table.hline(),
      ..cells.flatten(),
      table.hline(),
    )
  )]
}
```

図 8　三線表を組む関数 ftab

(ftab の例)

```
#ftab([測定結果],
  ([条件], [電圧 [V]]),
  (([A], [1.23]), ([B], [2.34])),
) <tab-ftab>

@tab-ftab を参照。
```

表 5　測定結果

条件	電圧 [V]
A	1.23
B	2.34

表 5 を参照。

共通の書式を関数にまとめておけば，テンプレートとして使い回せる。本書も共通の書式を `common.typ` にまとめている。公開されているテンプレートは Typst Universe で探せる[6]。

11. ソースコードの掲載

ソースコードは言語を指定して貼るだけで，等幅フォントで組まれ，言語に合わせて色分けもされる。パッケージ codelst を使うと[7]，図 9 のように，枠で囲み，行番号を付けて図として載せられる。

```python
data = [1.23456, 2.5, 3.14159]
for x in data:
    print(round(x, 3))
```

図 9　ソースコードの掲載例 (Python)

長いプログラムはファイルを直接読み込むと便利である。図 10 のように，行の範囲を指定すれば，必要な部分だけを本文に載せられる。ファイル全体は付録に載せ，本文では要点の数行だけを示す，という使い分けができる。ソースを修正しても，レポート側の掲載内容は自動で更新される。

```
#sourcefile(read("work4.ino"), file: "work4.ino",
  lang: "cpp", showrange: (42, 49))
```

図 10　ファイルの一部だけを載せる

12. 引用と参考文献

参考文献のデータは BibTeX 形式の `.bib` ファイルか，Hayagriva と呼ばれる専用の記法の YAML ファイルで書く。図 11 は Hayagriva 形式の例である。書式は CSL というスタイルファイルで選べ，学会や学科の書式に合わせて切り替えられる。本文では `@radar` のように引用すると，引用の順に番号が付き，末尾の文献一覧も自動で作られる。図 12 に，読み込みの書き方を示す。

```
radar:
  type: article
  title: "RADAR: An In-Building RF-based User Location and Tracking
System"
  author: ["Bahl, Paramvir", "Padmanabhan, Venkata N."]
  date: 2000
  page-range: 775-784
  parent:
    type: proceedings
    title: Proceedings of IEEE INFOCOM 2000
    volume: 2
```

図 11　参考文献データの例 (Hayagriva 形式)

```
位置推定には信号強度が使われる @radar。

#bibliography("ref.yaml", style: "ieee")
```

図 12　参考文献の指定

本書の参考文献は学科の書式[6]に合わせて，自作の関数で組んでいる。

13. 適用範囲と制約

表 6 に，Typst が向く場面と向かない場面を示す。この表は先に紹介した `ftab` で組んでいる。

[6]著者：タイトル，サイト名，URL，閲覧日の順に書く。

表 6　Typst が向く場面と向かない場面

向く	向かない・注意
実験レポート・課題・配布資料	縦書き・ルビ (外部パッケージで対応)[8]
数式や図表が多い文書	学会の投稿規定がTEX前提のとき
Git で履歴を管理したいとき	日本語フォントと `lang` の指定が必要
同じ書式を何度も使い回すとき	0.x 系で，書き方が変わることがある

実験レポートや課題，配布資料のように，PDF で提出する文書で，数式や図表が多いときは，Typst が向いている。書式をテンプレートにまとめて何度も使うときや，Git で履歴を管理するときにも向いている。

一方，縦書きやルビが必要な文書には向かない。標準機能になく，外部パッケージで対応することになる[8]。また，学会の投稿規定がTEXのテンプレートを前提にしているときは，TEXを使う方が確実である。日本語のフォントと `lang` を指定する必要がある点と，まだ 0.x 系で，バージョンによって書き方が変わることがある点にも注意が必要である。長く使う文書では，使った Typst とパッケージのバージョンを記録しておくとよい[9]。また，日本語の情報やテンプレートはLATEXよりまだ少ない。提出の形式が Word 指定のときは，Word を使う。

14. 導入方法

Typst の使い方は Web アプリ，コマンドラインツール，VS Code の 3 通りある。表 7 に，環境ごとの入れ方を示す。迷ったときは，インストール不要の Web アプリから始めるのがよい。手元に入れる場合，`typst` 本体は 1 つの実行ファイルで，LATEXのように大きな環境を用意する必要はない。

導入できたかどうかは，`typst --version` でバージョンを表示して確かめる。VS Code の拡張機能は拡張機能の画面で Tinymist Typst を検索して入れる[7]。Tinymist のプレビューは PDF ではなく独自に描画した画面なので，提出前には出力した PDF を PDF ビューアで開いて確かめる[8]。学校のパソコンのように，ソフトを入れられない環境では，Web アプリを使えばよい。パッケージは使うときに自動でダウンロードされるので，事前の準備は要らない。

表 7　環境ごとの Typst の入れ方

環境	入れ方	備考
Web アプリ	typst.app でアカウントを作る	インストール不要[10]
Windows	`winget install --id Typst.Typst`	winget の標準的な方法
Mac	`brew install typst`	Homebrew を使う
Linux	`sudo pacman -S typst` (Arch) `sudo snap install typst`	OS のパッケージ管理を使う
実行ファイル	GitHub から入手してパスに置く[1]	パッケージ管理を使わない方法
どの環境でも	`cargo install --locked typst-cli`	`cargo` が必要，ビルドが遅い
VS Code	Tinymist Typst を入れる[3]	ソースの横でプレビューできる

[7] コマンドでは `code --install-extension myriad-dreamin.tinymist` で入れられる。
[8] Tinymist にはスペルチェックの機能もないので，誤字は別の方法で確かめる。

15. 最初の 1 ページの作成手順

Step 1. **Web 版で試す** ブラウザで typst.app を開き，アカウントを作る。プレビュー付きのエディタが使え，インストールは要らない[10]。まずはここで，数行書いてみるとよい。

Step 2. **ローカルに入れる** 表 7 の方法で `typst` を入れる。VS Code を使うなら，拡張機能 Tinymist Typst も入れておく。

Step 3. **日本語フォントを指定して書く** 図 13 のように先頭に設定を書き，あとは本文を書く。フォント名は OS に入っているものを指定する。使えるフォントは `typst fonts` で確認できる。日本語の明朝体の例は Windows の Yu Mincho，Mac の Hiragino Mincho ProN，Linux の Noto Serif CJK JP である。

```
#set text(font: "Harano Aji Mincho", lang: "ja")
#set page(paper: "a4")

= はじめてのレポート
Typst で書いている。数式は $E = m c^2$ である。
```

図 13　最初の 1 ページ

16. 基本的なコマンド

図 14 に，よく使うコマンドを示す。`typst compile` は PDF を 1 回作る。`typst watch` はファイルを保存するたびに自動で再コンパイルする。`typst fonts` は使えるフォントの一覧を表示するので，日本語フォントの名前を調べるときに使う。VS Code で Tinymist を使う場合は，プレビューを開いておけば，`typst watch` と同じように，更新が自動で反映される。

```
typst compile report.typ   # PDFを1回作る
typst watch report.typ     # 保存のたびに自動で再コンパイル
typst fonts                # 使えるフォントの一覧
```

図 14　よく使うコマンド

17. 関連資料

書き方の詳しい説明は公式ドキュメントにあり，テンプレートやパッケージは Typst Universe で探せる[6,10]。VS Code で使う拡張機能の案内は Tinymist のページにある[3]。図 15 の QR コードから，それぞれのページに進める。

公式ドキュメント　　テンプレート集　　Tinymist(VS Code)

図 15　参考リンク

18. おわりに

筆者も実験レポートをpLaTeXからLuaLaTeXへ移し，現在はTypstで書いている。本書もTypstで作成した。

筆者がTypstを使い始めたのは2025年10月で，当時のバージョンは0.13.1だった。それから1年で，0.14ではタグ付きPDFやPDFの画像としての読み込み，文字単位の両端揃えが，0.15では可変フォントへの対応や和欧文間の間隔の修正などが加わった。Typstは今も発展を続けている。使う人が増えれば，不具合の報告やパッケージ，日本語の情報も増え，さらに使いやすくなる。

Typstはすべてを覚えてから使い始める必要はない。`=`で見出し，`$`で数式が書けることを知っていれば，レポートの骨組みは書ける。分からない点は書きながら公式ドキュメントで調べれば足りる。

まずはtypst.appで，次の実験レポートの「目的」だけでも書いてみてほしい。入力に合わせてプレビューが更新される感覚は，実際に触れると分かりやすい。合わなければ，従来の道具に戻ればよい。それも一つの比較の結果である。

参考文献

[1] Typst：typst/typst, GitHub, https://github.com/typst/typst (閲覧日 2026 年 10 月 25 日)

[2] Thoughtworks：Typst，Technology Radar，https://www.thoughtworks.com/en-ca/radar/tools/typst (閲覧日 2026 年 10 月 25 日)

[3] Myriad-Dreamin：tinymist, GitHub, https://github.com/Myriad-Dreamin/tinymist (閲覧日 2026 年 10 月 25 日)

[4] rein：Typstで実験レポートを書いた話, Zenn, https://zenn.dev/hanbaagu1/articles/mma-adc2025-day23 (閲覧日 2026 年 10 月 25 日)

[5] KZNS :cjk-unbreak, Typst Universe, https://typst.app/universe/package/cjk-unbreak (閲覧日 2026 年 10 月 25 日)

[6] Typst：Typst Universe, Typst, https://typst.app/universe/ (閲覧日 2026 年 10 月 25 日)

[7] jneug：codelst, Typst Universe, https://typst.app/universe/package/codelst/ (閲覧日 2026 年 10 月 25 日)

[8] 株式会社一創：Typstで縦書き・ルビはできる？和文組版の設定と使い方【0.15.1で検証】, 一創 Tech Blog, https://www.issoh.co.jp/tech/details/9859/ (閲覧日 2026 年 10 月 25 日)

[9] 株式会社一創 :Typst（タイプスト）とは？LaTeXとの違い・使い方・料金を初心者向けに解説【0.15対応】, 一創 Tech Blog, https://www.issoh.co.jp/tech/details/5507/ (閲覧日 2026 年 10 月 25 日)

[10] Typst：Typst Documentation, Typst, https://typst.app/docs/ (閲覧日 2026 年 10 月 25 日)